Genetic Algorithms

Rui Mendes

Introduction

Characteristics

- Inspired in living systems
- Borrow ideas from evolution

Examples

- Genetic Algorithms
- Genetic Programming

Genetic Algorithm

- Optimization algorithm
- Inspired in natural selection
- Uses a population of individuals
- Each individual encodes a solution
- There are three operators:
  1. Selection
  2. Crossover
  3. Mutation

Encoding

- Process that represents a solution in a given alphabet
- There are several encodings, e.g.:
  - binary
  - integer
  - real numbers
  - permutations
- Usually, they are linear sequences of values (chromossomes composed of genes)

Creating new solutions

- The genetic operators for creating new solutions are:
  - Recombination or Crossover
  - Mutation

Crossover

- Combines several solutions
- Usually, it takes two parents and creates two offspring
- Usual methods:
  - One point crossover
  - Two point crossover
  - Uniform crossover

One point crossover

- A cutting point is randomly selected
- Each solution takes a part from each parent

Parents

010|110
011|101

Offspring

010|101
011|110

Two point crossover

- Two cutting points are randomly selected
- The central parts are exchanged

Parents

10|11|0
01|10|1

Offspring

10|10|0
01|11|1

Uniform crossover

- The value of the first parent is randomly copied to one of the offspring
- The value of the second parent is copied to the other one

Parents

10110
01101

Offspring

00100
11111

Mutation

- A position is randomly selected inside the string
- The value is randomly selected among the possible values

Before

10110

After

10010

Fitness

- Quantifies the quality of a solution
- It is usually a *floating point number*
- In order to compare the quality of two solutions, one simply has to compare the fitness values

Selection

- Process for choosing which individuals will generate offspring or will be copied into the new population
- It usually selects individuals:
  - According to their fitness;
  - By partial or total ordering of the solutions
  - By stochastic choice of the individuals according to their fitness

Fitness proportionate selection

- The probability of selecting an individual is proportional to their fitness
- If it is a maximization problem, the probability of selecting an individual is $p_i = \frac{f_i}{\sum_i f_i}$

Selection by ordering

- If using total ordering, the individuals are ranked according to their fitness and the top n individuals are chosen
- In a partial ordering, several individuals are chosen at random and they are ordered (it's called *tournament selection*)

Genetic Algorithm outline

1. Initialize the population
2. While not finished
    1. Choose individuals
    2. Apply genetic operators
    3. Insert offspring into the next population

Generational scheme

- A new population is created in each iteration
- Parents are chosen from the current population
- Offspring are inserted into the new population
- The old population is discarded

Generational scheme

1. Initialize population
2. While not finished
    1. Evaluate individuals
    2. While the new population is not complete
        1. Select two individuals
        2. Apply genetic operators
        3. Insert offspring into the new population

Steady state scheme

- There are not generations
- Several individuals are randomly chosen for the tournament
- The best individuals produce offspring using the genetic operators
- The offspring substitute the worse individuals

Steady state scheme

1. Initialize population
2. While not finished
    1. Choose 4 individuals
    2. Rank the individuals
    3. Apply genetic operators to the best 2 individuals
    4. The offspring substitute the worst 2 individuals

Representations

- There can be several solution representations
- Each representation has several advantages and disadvantages
- There are several possibilities:
  - Integer
  - Real
  - Ordered

Integer representation

- Each individual is a sequence of integers, e.g.,

$$2 \quad 1 \quad 2 \quad 3 \quad 7$$

- Crossover operators defined thus far can be used
- There can be new crossover and mutation operators

Random value mutation

$$2\,1\,2\,3\,7 \rightarrow 2\,5\,2\,3\,7$$

Next element mutation

$$2\,1\,2\,3\,7 \rightarrow 2\,2\,2\,3\,7$$

Real representation

- Each individual is a sequence of real numbers, e.g.,

$$2.3 \quad -7.4 \quad 1.9 \quad 0.1$$

- There are specific crossover operators:
  - Arithmetic crossover
  - Linear crossover
  - Blend crossover
- And mutation operators:
  - Uniform mutation
  - Gaussian mutation
  - Cauchy modification

Arithmetic crossover

- Given two parents, $\vec{P}_1$ and $\vec{P}_2$
- Create 2 solutions:

  $\vec{S}_1 : \lambda \cdot \vec{P}_1 + (1 - \lambda) \cdot \vec{P}_2$

  $\vec{S}_2 : (1 - \lambda) \cdot \vec{P}_1 - \lambda \cdot \vec{P}_2$
- With λ usually being 0.6

Linear crossover

- Given two parents, $\vec{P}_1$ and $\vec{P}_2$
- Create 3 solutions:

  $\vec{S}_1 : 0.5 \cdot \vec{P}_1 + 0.5 \cdot \vec{P}_2$

  $\vec{S}_2 : 1.5 \cdot \vec{P}_1 - 0.5 \cdot \vec{P}_2$

  $\vec{S}_3 : 0.5 \cdot \vec{P}_1 + 1.5 \cdot \vec{P}_2$
- Choose the two best solutions among $\vec{P}_1$, $\vec{P}_2$, $\vec{S}_1$, $\vec{S}_2$ and $\vec{S}_3$ to become the children

Blend crossover

- Given two parents, x_1 and x_2 where $x_1 < x_2$
- Create solutions using:

$$U[x_1 - \alpha \cdot (x_2 - x_1), x_2 + \alpha \cdot (x_2 - x_1)]$$

- α is usually chosen to be 0.5.

Uniform mutation

- A random mutation where one or more positions are modified
- Modified positions are randomly generated within the entire range
- The modified position has no relationship with the original value
- $x_{new} = U[\min, \max]$

Random modification

- A random mutation where one or more positions are modified
- Modified positions are generated in the vicinity of the original value
- $x_{new} = x + U[-\epsilon, \epsilon]$ or
- $x_{new} = x \cdot (1 + U[-\epsilon, \epsilon])$

Gaussian mutation

- A random mutation where one or more positions are modified
- Modified positions are generated in the vicinity of the original value following the gaussian distribution
- $x_{new} = x + N(0, \sigma)$

Permutation based representations

- Each individual is a sequence of integers representing a permutation
- The integers are in the range $1, \ldots, n$ without repetitions
- Example:

$$2 \quad 1 \quad 5 \quad 4 \quad 3$$

- Genetic operators must preserve these constraints

Why ordinary crossover fails

Permutations need special treatment

- In a TSP-like chromosome, every city must appear **exactly once**.
- Ordinary two-point crossover can duplicate symbols and omit others.
- Such offspring are invalid tours.

Parent 1

`1 2 3 4 5 6 7 8`

Parent 2

`4 2 5 6 1 8 3 7`

If we blindly swap positions 4–6, we may get:

`1 2 3 6 1 8 7 8`

- City 1 appears twice.
- Cities 4 and 5 disappear.
- This is not a legal permutation.

Order preserving crossover

- Select a cutting position and copy the parts left of the cutting point directly to the corresponding offspring
- Copy the remaining values by using the order in the other parent
- Can also be used with two cutting points

Order preserving crossover

$2\ 1\ 5 \mid 3\ 4$ $\qquad$ $2\ 1\ 5 \mid 4\ 3$

$1\ 4\ 5 \mid 3\ 2$ $\qquad$ $1\ 4\ 5 \mid 2\ 3$

Partially Mapped Crossover (PMX)

Partially Mapped Crossover (PMX) is a two-point crossover for permutations.

- It copies a middle segment from one parent into the other.
- It builds a **partial mapping** from the two exchanged segments.
- It uses that mapping to replace duplicate symbols outside the segment.
- The result is always a valid permutation, assuming valid parent permutations.

PMX was introduced by Goldberg and Lingle for order-based problems such as the traveling salesman problem. [1][3]

The PMX algorithm

Four steps

Given two parent permutations P_1 and P_2:

1. Choose two crossover points i and j, with $i < j$.
2. Copy the segment $P_2[i:j]$ into offspring O_1, and $P_1[i:j]$ into offspring O_2.
3. Build a mapping from the exchanged segments, position by position.
4. Fill the remaining positions from the corresponding original parent, repairing conflicts through the mapping.

The middle segment is preserved **positionally**; the rest is repaired using the mapping. [1][3]

The PMX algorithm

Important convention

For offspring O_1:

- The middle segment comes from P_2.
- The outside positions are initially attempted from P_1.
- If a symbol from P_1 is already in the copied middle segment, follow the mapping until you find a usable replacement.

For offspring O_2, reverse the roles of P_1 and P_2.

Worked example

Parents and cut points

Use these parents:

$$P_1 = (3\ 4\ 8 \mid 2\ 7\ 1 \mid 6\ 5)$$

$$P_2 = (4\ 2\ 5 \mid 1\ 6\ 8 \mid 3\ 7)$$

The selected middle segments are:

- From P_1: $(2, 7, 1)$
- From P_2: $(1, 6, 8)$

This is the standard example used in descriptions of PMX. [3][10]

Worked example

Step 1: Exchange middle segments

Copy the opposite parent's middle segment into each child:

$$O_1 = (\times \ \times \ \times \mid 1\ 6\ 8 \mid \times \ \times)$$

$$O_2 = (\times \ \times \ \times \mid 2\ 7\ 1 \mid \times \ \times)$$

The middle segments are now fixed.

Worked example

Step 2: Build the mapping

Pair symbols at the same positions in the two middle segments:

Position	P_1	P_2
4	2	1
5	7	6
6	1	8

Therefore, the mapping is:

$$2 \leftrightarrow 1 \qquad 7 \leftrightarrow 6 \qquad 1 \leftrightarrow 8$$

This mapping is **bidirectional**.

Worked example

Step 3: Fill non-conflicting positions

For O_1, try to copy outside symbols from P_1:

$$P_1 = (3\ 4\ 8 \mid 2\ 7\ 1 \mid 6\ 5)$$

- Position 1: 3 is not in $\{1, 6, 8\}$, so place 3.
- Position 2: 4 is not in $\{1, 6, 8\}$, so place 4.
- Position 7: 6 **is** in the middle segment, so conflict.
- Position 8: 5 is not in $\{1, 6, 8\}$, so place 5.

$$O_1 = (3\ 4\ \times \mid 1\ 6\ 8 \mid \times\ 5)$$

Worked example

Step 4: Resolve the first conflict

At position 7, P_1 supplies 6, but 6 is already in O_1.
Use the mapping:

$$6 \leftrightarrow 7$$

So replace 6 with 7:

$$O_1 = (3\ 4\ \times \mid 1\ 6\ 8 \mid 7\ 5)$$

No conflict remains at this position.

Worked example

Step 5: Resolve the mapping chain

At position 3, P_1 supplies 8, but 8 is already in O_1.
Follow the mapping:

$$8 \leftrightarrow 1$$

But 1 is also already in O_1.
Continue:

$$1 \leftrightarrow 2$$

Now 2 is not yet in O_1, so place 2.

$$O_1 = (3\ 4\ 2 \mid 1\ 6\ 8 \mid 7\ 5)$$

This is the key PMX mechanism: follow the mapping **as many times as needed**.

Worked example

Completed first offspring

$$O_1 = (3\ 4\ 2 \mid 1\ 6\ 8 \mid 7\ 5)$$

Check:

- Every symbol from 1 to 8 appears exactly once.
- The middle segment $(1, 6, 8)$ came from P_2.
- The remaining symbols came from P_1, except where repaired by the mapping.

Worked example

Second offspring

For O_2, reverse the parents:

- Copy P_1's middle segment: $(2, 7, 1)$.
- Try to fill outside positions from P_2.
- Repair conflicts using the same mapping.

$$O_2 = (4\ 8\ 5 \mid 2\ 7\ 1 \mid 3\ 6)$$

Again, every symbol appears exactly once.

What the mapping really means

Mapping is not a lookup table for all symbols

The PMX mapping comes **only** from the exchanged middle segments.
In the example:

$$2 \leftrightarrow 1 \qquad 7 \leftrightarrow 6 \qquad 1 \leftrightarrow 8$$

- It does **not** say that every 2 in the chromosome must become 1.
- It is used only when an outside gene conflicts with a gene already placed in the middle segment.
- Outside positions without conflicts are copied directly from the original parent.

This distinction is crucial.

What the mapping really means

A conflict triggers a chain

Suppose O_1 contains middle segment $(1, 6, 8)$, and the outside gene from P_1 is 8.

- 8 conflicts because it is already present.
- Mapping says $8 \leftrightarrow 1$.
- But 1 is also present.
- Mapping says $1 \leftrightarrow 2$.
- 2 is absent, so use 2.

The repair path is:

$$8 \rightarrow 1 \rightarrow 2$$

PMX follows the chain until it reaches a symbol not already present in the offspring.

What the mapping really means

Why chains can occur

The mapping is built from paired positions:

P_1 segment		P_2 segment
2	$\leftrightarrow$	1
7	$\leftrightarrow$	6
1	$\leftrightarrow$	8

A symbol can appear on both sides of different pairs.

For example, 1 is:

- The right-hand partner of $2 \leftrightarrow 1$.
- The left-hand partner of $1 \leftrightarrow 8$.

That creates the potential chain:

$$8 \leftrightarrow 1 \leftrightarrow 2$$

Second example: longer chain

Parents

$$P_1 = (1\ 2 \mid 3\ 4\ 5\ 6 \mid 7\ 8)$$

$$P_2 = (2\ 7 \mid 5\ 8\ 4\ 1 \mid 6\ 3)$$

The middle segments are:

- From P_1: $(3, 4, 5, 6)$
- From P_2: $(5, 8, 4, 1)$

The initial offspring are:

$$O_1 = (\times\ \times \mid 5\ 8\ 4\ 1 \mid \times\ \times)$$

$$O_2 = (\times\ \times \mid 3\ 4\ 5\ 6 \mid \times\ \times)$$

Second example: longer chain

Mapping

Pair the middle segments positionally:

$$3 \leftrightarrow 5 \quad 4 \leftrightarrow 8 \quad 5 \leftrightarrow 4 \quad 6 \leftrightarrow 1$$

Notice that:

- 4 appears in two pairs.
- 5 appears in two pairs.

So the mapping contains a cycle-like structure:

$$3 \leftrightarrow 5 \leftrightarrow 4 \leftrightarrow 8$$

and separately:

$$6 \leftrightarrow 1$$

Second example: longer chain

Fill O_1

Try outside genes from P_1:

$$P_1 = (1\ 2 \mid 3\ 4\ 5\ 6 \mid 7\ 8)$$

- Position 1: gene 1 conflicts with the middle segment.
- Position 2: gene 2 does not conflict.
- Position 7: gene 7 does not conflict.
- Position 8: gene 8 conflicts with the middle segment.

After direct placements:

$$O_1 = (\times\ 2 \mid 5\ 8\ 4\ 1 \mid 7\ \times)$$

Second example: longer chain

Resolve position 1

At position 1, P_1 supplies 1, but 1 is already in O_1.
Use the mapping:

$$1 \leftrightarrow 6$$

Symbol 6 is not in O_1, so place 6:

$$O_1 = (6\ 2 \mid 5\ 8\ 4\ 1 \mid 7\ \times)$$

Second example: longer chain

Resolve position 8

At position 8, P_1 supplies 8, but 8 is already in O_1.
Use the mapping:

$$8 \leftrightarrow 4$$

But 4 is already in O_1. Continue:

$$4 \leftrightarrow 5$$

But 5 is already in O_1. Continue:

$$5 \leftrightarrow 3$$

Symbol 3 is not in O_1, so place 3:

$$O_1 = (6\ 2 \mid 5\ 8\ 4\ 1 \mid 7\ 3)$$

Second example: longer chain

Result

$$O_1 = (6\ 2 \mid 5\ 8\ 4\ 1 \mid 7\ 3)$$

The repair path was:

$$8 \rightarrow 4 \rightarrow 5 \rightarrow 3$$

This demonstrates that PMX may require **multiple mapping steps**, not merely one substitution.

Properties and pitfalls

What PMX preserves

PMX tends to preserve:

- **Absolute positions** in the copied middle segment.
- Some positional information outside the segment.
- A mixture of material from both parents.
- Feasibility of the permutation.

It does **not** primarily preserve relative order across the whole chromosome; that is closer to Order Crossover (OX). [3][5]

Properties and pitfalls

Common misunderstanding 1

Wrong: "If symbol a maps to b, replace every a in the child with b."
Correct: Use the mapping only when an outside gene conflicts with a symbol already placed in the copied middle segment.
If there is no conflict, copy the original parent gene unchanged.

Common misunderstanding 2

Wrong: "The mapping is directed from P_1 to P_2."
Correct: The mapping is bidirectional.
For O_1, you often follow mappings in one direction; for O_2, you use the same pairs but with the parents' roles reversed.

Properties and pitfalls

Common misunderstanding 3

Wrong: "A conflict is always repaired in one step."
Correct: A conflict may require a chain:

$$a \to b \to c \to d$$

Stop when the candidate symbol is not already present in the offspring.

PMX: Implementation checklist

For each offspring:

1. Initialize all positions as empty.
2. Copy the opposite parent's middle segment.
3. Record mappings from paired middle-segment positions.
4. For each outside position, take the gene from the original parent.
5. If that gene is absent, place it.
6. If it conflicts, follow the mapping until you find an absent gene.
7. Place that repaired gene.
8. Verify that the output is a permutation.

PMX in one slide

PMX = two-point segment exchange + partial mapping + conflict repair

- The middle segment is copied directly.
- The mapping is built only from that segment.
- Outside genes are copied unless they conflict.
- Conflicts are resolved by following the mapping chain.
- The final children are valid permutations.

PMX: Remember the example

$$P_1 = (3\ 4\ 8 \mid 2\ 7\ 1 \mid 6\ 5)$$

$$P_2 = (4\ 2\ 5 \mid 1\ 6\ 8 \mid 3\ 7)$$

$$O_1 = (3\ 4\ 2 \mid 1\ 6\ 8 \mid 7\ 5)$$

The difficult part was resolving 8:

$$8 \rightarrow 1 \rightarrow 2$$

That chain is the essence of PMX.

Insertion mutation

- Choose two positions at random
- Move the second position to follow the first

Insertion mutation

$$2\ 1\ 5\ 3\ 4 \rightarrow 2\ 1\ 3\ 5\ 4$$

Swap mutation

- Choose two positions at random
- Swap their positions

Swap mutation

$$2\ 1\ 5\ 3\ 4 \rightarrow 2\ 3\ 5\ 1\ 4$$

Inversion mutation

- Choose two positions at random
- Invert the values between them

Inversion mutation

$$2\ 1\ 5\ 3\ 4 \rightarrow 2\ 3\ 5\ 1\ 4$$

Scramble mutation

- Pick a random subset of genes
- Randomly change the position of the values between them
- The subset need not be contiguous

Scramble mutation

$$2\ 1\ 7\ 5\ 3\ 6\ 4 \rightarrow 2\ 7\ 3\ 6\ 5\ 1\ 4$$